

Graph Neural Networks (GNNs)

Chapter 17 Graph Neural Network

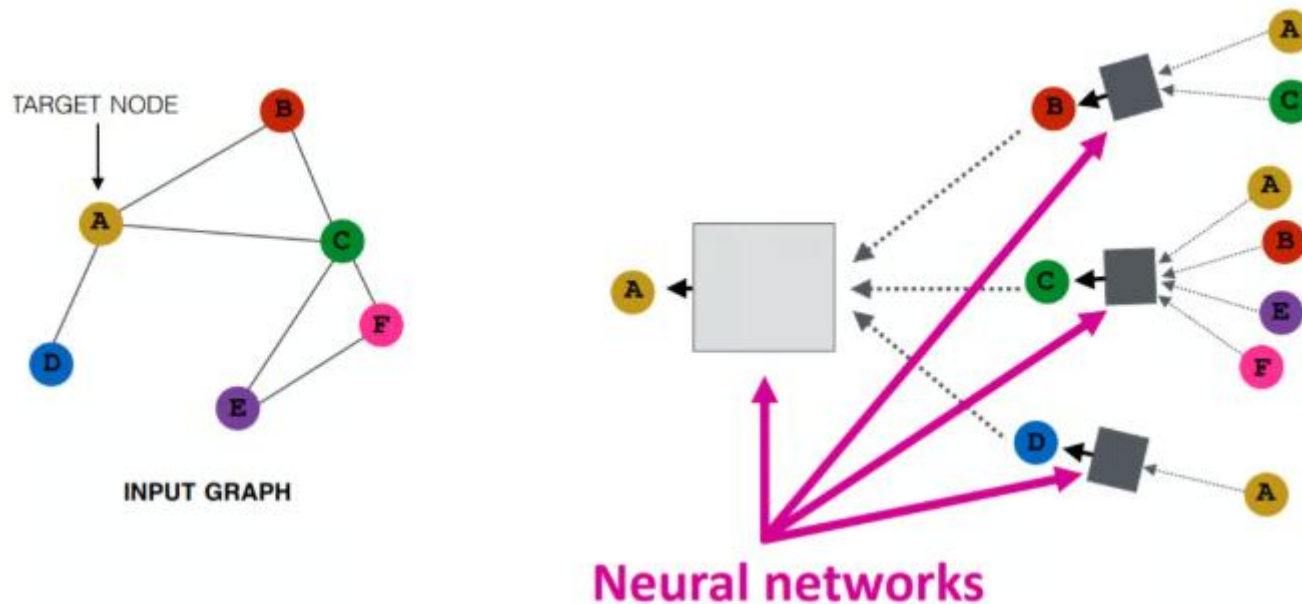
What is a Graph?

A Graph is the type of data structure that contains nodes and edges. A node can be a person, place, or thing, and the edges define the relationship between nodes. The edges can be directed and undirected based on directional dependencies.

Graph Neural Network (GNN)

- Graph Neural Networks are special types of neural networks capable of working with a graph data structure. They are highly influenced by Convolutional Neural Networks (CNNs) and graph embedding. GNNs are used in predicting nodes, edges, and graph-based tasks.
- **CNNs** are used for image classification. Similarly, GNNs are applied to graph structure (grid of pixels) to predict a class.
- **Recurrence Neural Networks** are used in text classification. Similarly, GNNs are applied to graph structures where every word is a node in a sentence.

GNNs were introduced when Convolutional Neural Networks failed to achieve optimal results due to the arbitrary size of the graph and complex structure.



The model aggregates message from A's local graph neighbor(B,C,D) and in turn the message are coming from the neighbors are based on the info. Aggregated from their respective neighborhoods.

Types of Graph Neural Networks

Tasks

- **Graph Classification:** we use this to classify graphs into various categories. Its applications are social network analysis and text classification.
- **Node Classification:** this task uses neighboring node labels to predict missing node labels in a graph.
- **Link Prediction:** predicts the link between a pair of nodes in a graph with an incomplete adjacency matrix. It is commonly used for social networks.
- **Community Detection:** divides nodes into various clusters based on edge structure. It learns from edge weights, and distance and graph objects similarly.
- **Graph Embedding:** maps graphs into vectors, preserving the relevant information on nodes, edges, and structure.
- **Graph Generation:** learns from sample graph distribution to generate a new but similar graph structure.

Graph Machine Learning

- It refers to technique that apply machine learning on graph structured data where data is represented as:
- $G=(V,E,X)$ where V is the vertex, E is the edges and X is the feature of node.
- Classical Machine Learning are independent samples but graph data points are independent, Graph captures these dependencies.

Adjacency Matrix

$$A_{ij} = \begin{cases} 1 & \text{if edge exist} \\ 0 & \text{otherwise} \end{cases}$$

Graph neural network are deep learning model designed to operate on graph structured data where relationships between entities are represented as:

$$G=(V,E,X)$$

Each node learns by aggregating info. From its neighbor.

Algorithm

At layer k , for node v

Initialization: Assign node features

Message Computation: collect neighbor features

$$m_v = \sum_{u \in N(v)} h^{(k-1)}$$

Aggregation(sum/mean/max):

$$m_v = \text{AGG}(h^{(k-1)})$$

Updated: Apply weight and activation function

General GNN Equation

$$h_i^{(l+1)} = \sigma \left(\text{AGG}(\{ h_j^{(l)} \mid j \in \mathcal{N}(i) \}) \cdot W^{(l)} \right)$$

- $h_i^{(l)}$ → Feature (embedding) of node i at layer l
- $\mathcal{N}(i)$ → Set of neighbors of node i
- **AGG** → Aggregation function (mean, sum, max)
- $W^{(l)}$ → Trainable weight matrix
- σ → Activation function (ReLU, sigmoid, etc.)

Node Classification(Graph Convolution Network)

Chapter 17 Graph Neural Network

Node classification

Node classification objective in **Graph Neural Networks (GNNs)** to **predict the label (category/class)** of each node in a graph using both:

- the node's own features, and
- the connecting nodes

Example: *In a social network graph. Let each node is a person and edge is the friendship and the task is to classify the person is a engineer, student or teacher depending on own profile along with the connection.*

Working of Node Classification

Step 1: Initial Features

Each node has feature X

Step 2: Message Passing

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in N(i)} h_j^{(l)} W \right)$$

Step 3: Final Prediction

$$y_i = \text{Softmax}(h_i)$$

Each node gets its own class

Example

Step 1: Compute AX

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

$$AX = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 1 & 2 \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

Step 2: Compute AXW

$$W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$AXW = \begin{bmatrix} 4 & 6 \\ 8 & 12 \\ 7 & 10 \end{bmatrix}$$

Step 3: Apply ReLU

$$H = \begin{bmatrix} 4 & 6 \\ 8 & 12 \\ 7 & 10 \end{bmatrix}$$

Step 4: Node-wise Classification

Apply Softmax:

Example for Node 0:

$$\text{Softmax}([4, 6]) = \left[\frac{e^4}{e^4 + e^6}, \frac{e^6}{e^4 + e^6} \right]$$

Graph Classification

- **Graph classification** is the task of assigning a **single label to an entire graph**, instead of individual nodes.

Example:

Molecule Classification

Graph → Molecule

Label → Toxic / Non-toxic

Social Network

Graph → Group of users

Label → Fraud / Normal

Other Examples:

- Drug discovery
- Protein classification
- Fraud detection
- Brain network analysis

Working of Graph Classification

Step 1: Node Embedding

Each node gets features via GNN:

$$h_i^{(l+1)} = \sigma(\text{AGG}(h_j^{(l)})W)$$

Step 2: Graph Readout (Pooling)

Combine all node features:

$$h_G = \text{READOUT}(h_1, h_2, \dots, h_n)$$

Common function: Mean pooling, sum pooling or Max pooling

Step 3: Classification

$$y = \text{Softmax}(h_G)$$

Graph Classification Numerical Example

- Suppose after GNN message passing, node embeddings are:

$$H = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 6 & 7 \end{bmatrix}$$

3. Max Pooling

$$h_G = [\max(2, 4, 6), \max(3, 5, 7)]$$

$$h_G = [6, 7]$$

1. Mean Pooling

$$h_G = \left[\frac{2 + 4 + 6}{3}, \frac{3 + 5 + 7}{3} \right]$$

$$h_G = [4, 5]$$

2. Sum Pooling

$$h_G = [2 + 4 + 6, 3 + 5 + 7]$$

$$h_G = [12, 15]$$

Final Classification Step:

Assume dense layer weight:

$$W = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$$

Using mean pooled vector:

$$h_G = [4, 5]$$

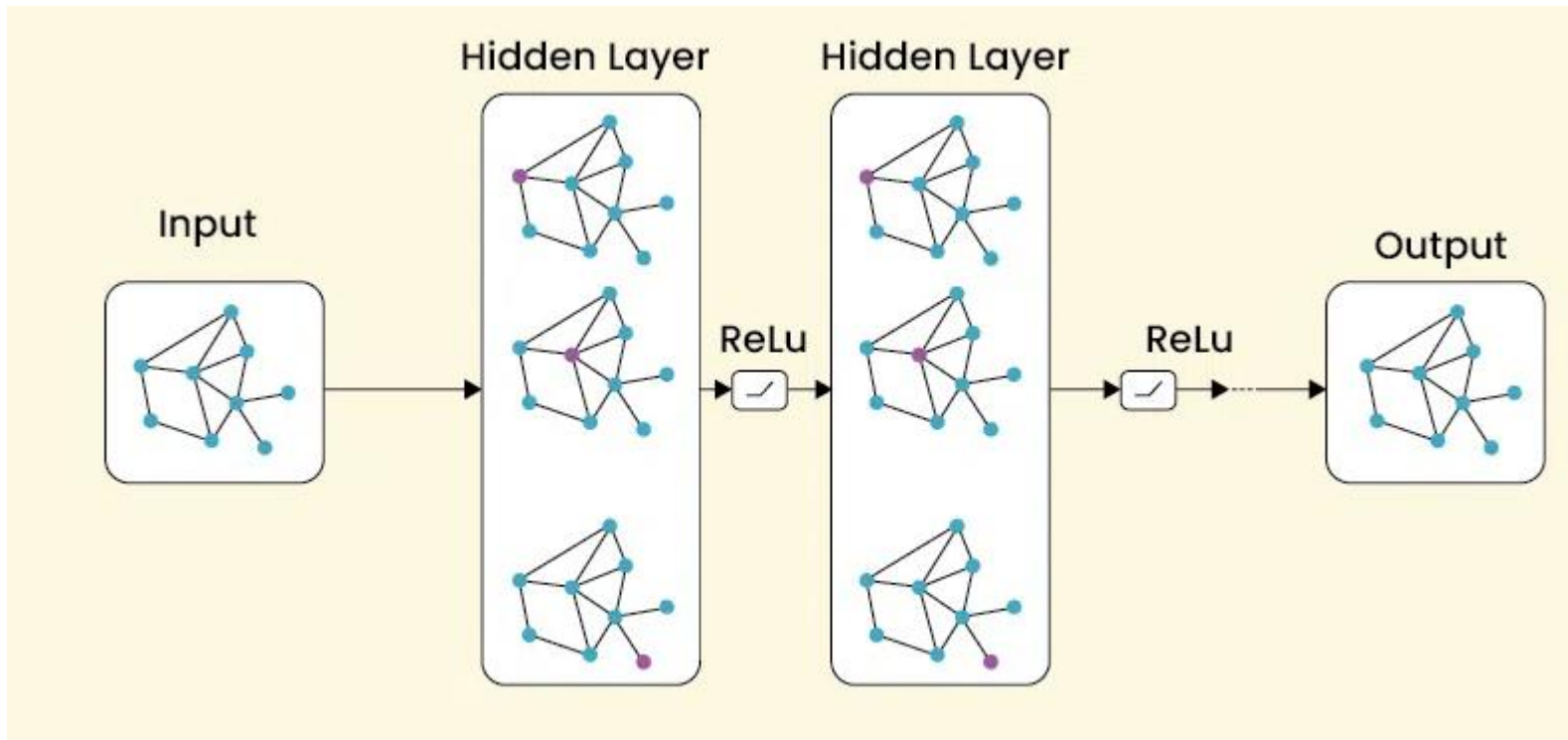
$$Z = h_G W$$

$$Z = [4, 5] \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$$

$$Z = [19, 13]$$

Since $19 > 13$ graph belong to class 0

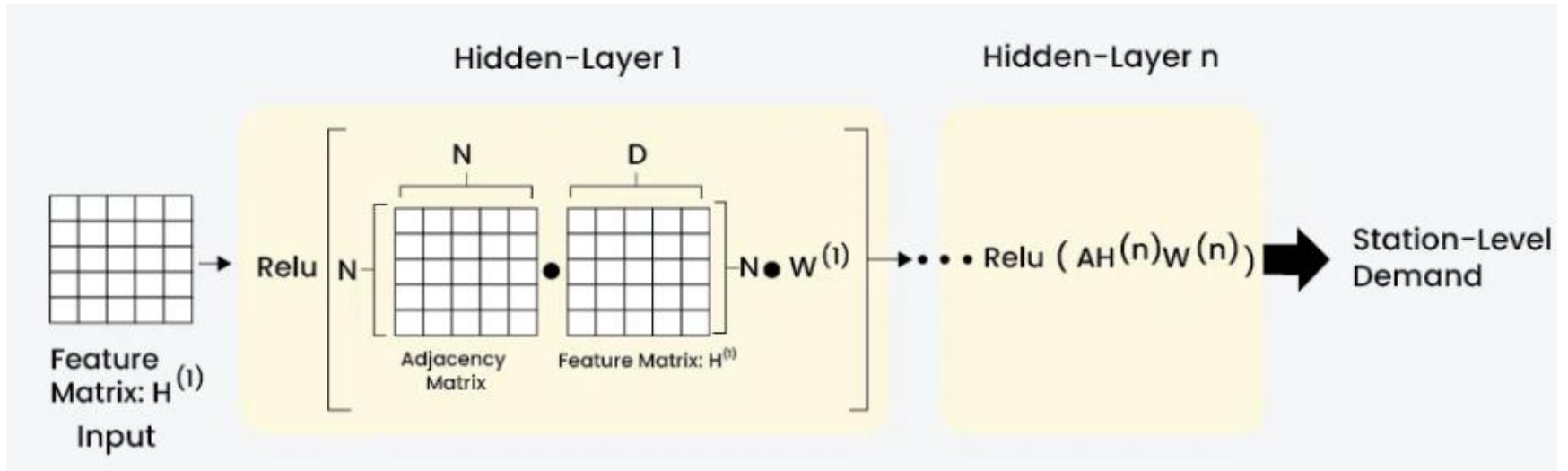
Graph Convolutional Networks (GCNs)



Graph Convolutional Networks

- Graph Convolutional Networks (GCNs) are a type of neural network designed to work directly with graphs.
- The primary goal of GCNs is to learn node embeddings, which are vector representations of nodes that capture the graph's structural and feature information.

Architecture of GCNs



Input Layer: The input layer initializes the node features, usually from raw data or pre-trained embeddings.

Graph $G=(V,E)$

Node feature matrix:

- $X \in \mathbb{R}^{N \times F}$

Hidden Layers: Hidden layers perform the graph convolution operations, progressively aggregating and transforming node features.

- **Graph Convolutional Layers:** These layers perform the convolution operation on the graph. Each layer updates the feature representation of a node by aggregating the features of its neighbors.
- **Activation Functions:** Non-linear functions such as ReLU are applied to the output of each convolutional layer to introduce non-linearity into the model.
- **Pooling Layers:** These layers reduce the dimensionality of the graph by merging nodes, which helps in capturing hierarchical structures.

Operation Performed in Hidden Layer

$$H^{(l+1)} = \sigma \left(\hat{A} H^{(l)} W^{(l)} \right)$$

Where:

- $H^{(l)}$: Node features at layer l
- $W^{(l)}$: Trainable weight matrix
- $\hat{A}$: Normalized adjacency matrix
- σ : Activation function (ReLU)

The Normalized Adjacency matrix is computed using: $\hat{A} = D^{-1/2} A D^{-1/2}$

Node Graph Example

Let a graph structure with the Adjacency Matrix is defined as

A =

0	1	0
1	0	1
0	1	0

And let the node connections are:

Node1->Node 2

Node 2->Node 1, Node 3

Node 3-> Node 2

By adding the self loop(identity matrix) to the adjacency matrix, the final output will be:

$$A' = A + I =$$

0	1	0
1	0	1
0	1	0

 $+$

1	0	0
0	1	0
0	0	1

 $=$

1	1	0
1	1	1
0	1	1

Degree Matrix

- Degree Matrix is represented as:

2	0	0
0	3	0
0	0	2

- Normalize Adjacency is

$$\hat{A} = D^{-1/2} A D^{-1/2}$$

$$D^{-1/2} = \begin{bmatrix} \frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} 0.707 & 0 & 0 \\ 0 & 0.577 & 0 \\ 0 & 0 & 0.707 \end{bmatrix}$$

After Multiplication:

$$\hat{A} = \begin{bmatrix} 0.5 & 0.408 & 0 \\ 0.408 & 0.333 & 0.408 \\ 0 & 0.408 & 0.5 \end{bmatrix}$$

This is basically message passing with aggregation.

The shortcut formula to find the adjacency

matrix is: $\widehat{A}_{ij} = \frac{A_{ij}}{\sqrt{D_{ij}}\sqrt{D_{jj}}}$

Let each nodes has two features as given:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \text{ and the weight matrix is } W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\text{then the value of } XW = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 4 & 6 \end{bmatrix}$$

Multiply with $\hat{A}$

$$Z = \hat{A} (XW)$$

$$\begin{bmatrix} 0.5 & 0.408 & 0 \\ 0.408 & 0.333 & 0.408 \\ 0 & 0.408 & 0.5 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 4 & 6 \end{bmatrix}$$

$$\begin{bmatrix} 1.724 & 2.632 \\ 3.039 & 4.596 \\ 3.224 & 4.632 \end{bmatrix}$$

Apply Activation (ReLU)

$$H = \text{ReLU}(Z)$$

$$H = \begin{bmatrix} 1.724 & 2.632 \\ 3.039 & 4.596 \\ 3.224 & 4.632 \end{bmatrix}$$

With Normalization:

- Equal contribution from all node is obtained.
- Keep is stable across all layer prevent exploding value
- Avoid bias toward high degree nodes
- Stabilizes training by improving convergence.
- It is similar to feature scaling.

Output Layer: The output layer produces the final node embedding or predictions, depending on the task (e.g., node classification, link prediction).

Fully Connected Layer: These layers used at the end of the network to perform classification.

Types of Graph Convolutional Networks (GCNs)

Spectral-based GCNs

Spectral-based GCNs are defined in the spectral domain using the graph Laplacian and Fourier transform. The convolution operation is performed by multiplying the graph signal with a filter in the spectral domain.

Key Model:

ChebNet

$$H = \sigma(U g_{\theta}(\Lambda) U^T X)$$

Spatial-based GCNs

Spatial-based GCNs perform convolution directly in the spatial domain by aggregating features from neighboring nodes.

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in N(i)} W h_j^{(l)} \right)$$

Key Models:

- **GraphSAGE**: Aggregates features from a fixed-size set of neighbors using mean, LSTM or pooling functions.
- **GAT (Graph Attention Network)**: Introduces an attention mechanism to assign different weights to the neighbors of a node based on their importance.

GRAPHSAGE(Graph **S**Amples and Aggre**G**ate**E**)

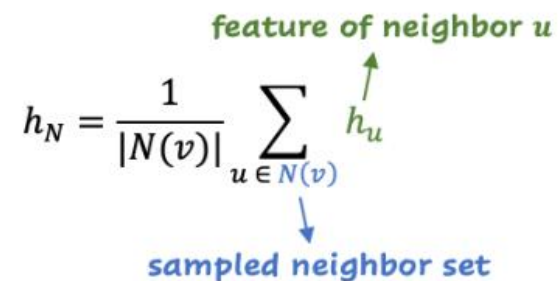
GraphSAGE makes training much faster and scalable. It does this by *sampling only a subset of neighbors*. For super large graphs it's computationally impossible to process all neighbors of a node

Sampling Neighbors

- With tabular data, sampling is easy. But with graphs, we cannot select random nodes. This can result in disconnected graphs, nodes without neighbors.
- So in graph we need to select a random fixed-size subset of neighbors. For example in a social network, you can sample 3 friends for each user (instead of all friends):

Aggregate Information

After the neighbor selection from the previous part, GraphSAGE combines their features into one single representation. There are multiple ways to do this (multiple *aggregation functions*). The most common types and the ones explained in the paper are *mean aggregation*, *LSTM*, and *pooling*.

$$h_N = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u$$


The diagram shows the formula for mean aggregation. The variable h_u is annotated with a green arrow pointing to it from the text "feature of neighbor u". The summation index $u \in N(v)$ is annotated with a blue arrow pointing to it from the text "sampled neighbor set".

LSTM aggregation uses an [LSTM](#) (type of neural network) to process neighbor features sequentially. It can capture more complex relationships, and is more powerful than mean aggregation.

Update Node Representation

After sampling and aggregation, the node *combines its previous features with the aggregated neighbor features*. Nodes will learn from their neighbors but also keep their own identity. Information can flow across the graph effectively.

The diagram shows the equation
$$h'_v = \sigma(W \cdot \text{CONCAT}(h_v, \text{AGGREGATE}(h_u, \forall u \in N(v))))$$
 with several annotations: a blue arrow points from the text "non-linear activation function" to the σ symbol; a green arrow points from the text "weight matrix" to the W symbol; a blue arrow points from the text "concatenate vectors" to the CONCAT function; and a green arrow points from the text "aggregation function of step 2" to the AGGREGATE function.

$$h'_v = \sigma(W \cdot \text{CONCAT}(h_v, \text{AGGREGATE}(h_u, \forall u \in N(v))))$$

Mathematical Formula

For node v :

Step 1: Aggregate neighbors

$$h_{N(v)}^{(k)} = \text{AGGREGATE}^{(k)} \left(\{h_u^{(k-1)}, \forall u \in N(v)\} \right)$$

Step 2: Combine with self node

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT}(h_v^{(k-1)}, h_{N(v)}^{(k)}) \right)$$

Step 3: Normalize

$$h_v^{(k)} = \frac{h_v^{(k)}}{\|h_v^{(k)}\|}$$

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class GraphSAGE(nn.Module):
    def __init__(self, in_feats, hidden,
out_feats):
        super().__init__()
        self.fc1 = nn.Linear(in_feats * 2,
hidden)#for concat operation
        self.fc2 = nn.Linear(hidden * 2, out_feats)

    def aggregate(self, neighbors):
        return torch.mean(neighbors, dim=1)

```

In_feats is the input feature dimension per node
Out_feats is the output dimensions

```

def forward(self, x, neighbor_features):
    h_neigh =
self.aggregate(neighbor_features)
    h = torch.cat([x, h_neigh], dim=1)
    h = F.relu(self.fc1(h))

    h_neigh2 =
self.aggregate(neighbor_features)
    h = torch.cat([h, h_neigh2], dim=1)
    h = self.fc2(h)

    return h

```

Link Prediction in Social Network Using GraphSAGE

Objective:

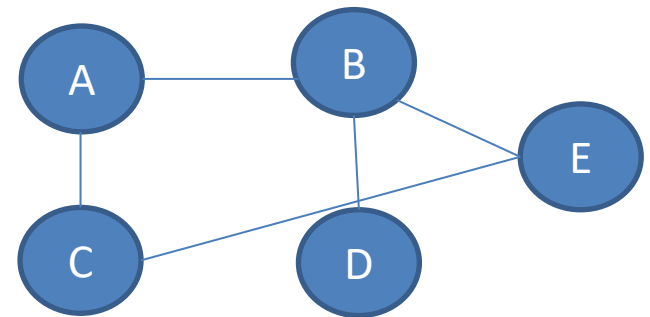
To predict whether two users should become friends.

Existing friendships:

$(A,B), (A,C), (B,D), (B,E), (C,E)$

Prediction target:

A-D?



Each user has features:[sports,music]

A: [1,0]

B: [1,1]

C: [0,1]

D: [1,0]

E: [0,1]

GraphSage Aggregation

GraphSAGE updates each node by combining:

Self feature+neighbor feature

$$h_v^{(k)} = \sigma \left(W^{(k)} [h_v^{(k-1)} || AGG(N(v))] \right)$$

For user A

Neighbors of A: {B, C}

$B=[1,1]$, $C=[0,1]$

Mean neighbor feature:

$$\begin{aligned} \text{AGG}(A) &= 1/2 * ([1,1] + [0,1]) \\ &= [0.5, 1] \end{aligned}$$

Combine with A: $[1, 0, 0.5, 1]$

For user D

Neighbors of D: {B}

$B=[1,1]$

Mean neighbor feature:

$$\begin{aligned} \text{AGG}(D) &= [1, 1] \\ &= [1, 1] \end{aligned}$$

Combine with D: $[1, 0, 1, 1]$

Let the weight matrix $w =$

1	0	1	0
0	1	0	1

$$h_A = \text{ReLU}(W \cdot [1, 0, 0.5, 1]^T) = [1.5, 1]$$

Similarly for D: $h_D = \text{ReLU}(W \cdot [1, 0, 1, 1]^T) = [2, 1]$

To Predict the link

$$\text{Score}(A, D) = h_A \cdot h_D$$

$$= [1.5, 1] \cdot [2, 1]$$

$$= (1.5 \cdot 2) + (1 \cdot 1) = 4$$

Convert to Propability $P(A, D) = \text{sigmoid}(4)$

$$= \sigma(4) = \frac{1}{1 + e^{-4}} \approx 0.982$$

So the model predicts:

A and D are likely to become friends.

Limitation of GraphSAGE

- Information Loss Due to Sampling important neighbors may be ignored
- Aggregation is simple as it treats all neighbor equally.
- No Edge feature utilization
- Over smoothing in deep layers as all node embedding become similar leads to poor prediction.
- Computational cost is high.

Graph Attention Network (GAT)

Unlike GCN / GraphSAGE:

- Not all neighbors are treated equally
- Important neighbors get **higher weight**

Core Idea

Instead of simple mean aggregation:

$$AGG = \frac{1}{|N(v)|} \sum h_u$$

GAT uses attention weights:

$$h'_v = \sigma \left(\sum_{u \in N(v)} \alpha_{vu} W h_u \right)$$

where:

α_{vu} = attention coefficient

W = weight matrix

σ = activation

Step 1: Linear Transformation

$$h'_u = Wh_u$$

Step 2: Compute Attention Scores

For nodes u and v :

$$e_{vu} = \text{LeakyReLU}(a^T [Wh_v || Wh_u])$$

Measures importance of neighbor u for node v .

Step 3: Normalize (Softmax)

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

Makes weights sum to 1

Step 4: Weighted Aggregation

$$h'_v = \sum_{u \in N(v)} \alpha_{vu} h'_u$$